

HashMap

- Java HashMap class implements the Map interface which allows us to store key and value pair, where keys should be unique.
- If you try to insert the duplicate key, it will replace the element of the corresponding key.
- It is easy to perform operations using the key index like updation, deletion, etc. HashMap class is found in the java.util package.
- HashMap in Java is like the legacy Hashtable class, but it is not synchronized.
- It allows us to store the null elements as well, but there should be only one null key.
- Since Java 5, it is denoted as HashMap<K,V>, where K stands for key and V for value.
- It inherits the AbstractMap class and implements the Map interface.

Points to remember

- Java HashMap contains values based on the key.
- Java HashMap contains only unique keys.
- Java HashMap may have one null key and multiple null values.
- Java HashMap is non synchronized.
- Java HashMap maintains no order.
- The initial default capacity of Java HashMap class is 16

Methods of HasMap

```
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;

public class Hash2 {
    public static void main(String[] args) {
        HashMap<Integer, String> hm1 = new HashMap<>();
        HashMap<Integer, String> hm2 = new HashMap<>();

        //(1)//
        /*public V put(K key, V value)
        {
            return putVal(hash(key), key, value, false, true);
        } */

        //Associates the specified value with the specified key in this map.
        //If the map previously contained a mapping for the key, the old
        //value is replaced.

        //return the previous value associated with key, or null if there was
        //no mapping for key.

        System.out.println(hm1.put(10, "JJS")); //null
        hm1.put(10, "JJS");
```

```

        hm1.put(15, "HDS");
        hm1.put(20, "CVM");
        hm1.put(30, "PAT");
        hm1.put(40, "DJU");
        hm1.put(60, "DJU");
        hm1.put(70, "DJU");

        System.out.println("hm1 is: " + hm1);
        //hm1 is: {20=CVM, 70=DJU, 40=DJU, 10=JJS, 60=DJU, 30=PAT, 15=HDS}
    /*_____*/

        //(2)//
        /*public Collection<V> values() */
        //return a view of the values contained in this map

        System.out.println(hm1.values());
        //[CVM, DJU, DJU, JJS, DJU, PAT, HDS]

        //(3)//
        /* public Set<K> keySet() */
        //return a set view of the keys contained in this map

        System.out.println(hm1.keySet());
        //[20, 70, 40, 10, 60, 30, 15]
    /*_____*/

        //(4)//
        /*public void putAll(Map<? extends K, ? extends V> m)*/

        //Copies all of the mappings from the specified map to this map.
        //These mappings will replace any mappings that this map had for
        //any of the keys currently in the specified map.

        hm2.putAll(hm1);
        System.out.println("hm2 is: "+hm2);
        //hm2 is: {20=CVM, 70=DJU, 40=DJU, 10=JJS, 60=DJU, 30=PAT, 15=HDS}
    /*_____*/

        //(5)//
        /*public boolean isEmpty() */
        //Returns true if this map contains no key-value mappings.
        //returns false if this map doesn't contains any key-value mappings.

        System.out.println(hm1.isEmpty()); //false

    /*_____*/

```

```

//(6)//
/* public V remove(Object key) */

// Removes the mapping for the specified key from this map if present.
// return the previous value associated with key, or null if there was
// no mapping for key.

System.out.println("Removed element is: "+hm1.remove(100));
//Removed element is: null

System.out.println(hm1);
//{20=CVM, 70=DJU, 40=DJU, 10=JJS, 60=DJU, 30=PAT, 15=HDS}

System.out.println("Removed element is: " + hm1.remove(20));
//Removed element is: CVM

System.out.println(hm1);
//{70=DJU, 40=DJU, 10=JJS, 60=DJU, 30=PAT, 15=HDS}

//(7)//
/*public boolean remove(Object key, Object value) */

//remove(key,value) method returns boolean true and removes pair if
available.
//remove(key,value) method returns boolean false if pair is not
available.

System.out.println("Removed element is: " + hm1.remove(10, "CVM"));
//Removed element is: false

System.out.println(hm1);
//{70=DJU, 40=DJU, 10=JJS, 60=DJU, 30=PAT, 15=HDS}

System.out.println("Removed element is: " + hm1.remove(10, "JJS"));
//Removed element is: true

System.out.println(hm1);
//{70=DJU, 40=DJU, 60=DJU, 30=PAT, 15=HDS}
/* _____ */

//(8)//
/* public int size() */
//Returns the number of key-value mappings in this map.

System.out.println(hm1.size());//5
System.out.println(hm1);
//{70=DJU, 40=DJU, 60=DJU, 30=PAT, 15=HDS}

```

```

        System.out.println( hm2.size());//7
        System.out.println(hm2);
        //{20=CVM, 70=DJU, 40=DJU, 10=JJS, 60=DJU, 30=PAT, 15=HDS}

/*_____*/

        //(9)//
        /*public V merge(K key, V value,BiFunction<? super V, ? super V, ? extends
V> remappingFunction) */
        System.out.println(hm1);
        //{70=DJU, 40=DJU, 60=DJU, 30=PAT, 15=HDS}

        hm1.merge(40, ",ARP", (v1,v2)->(v1+v2) );

        System.out.println(hm1);
        //{70=DJU, 40=DJU,ARP, 60=DJU, 30=PAT, 15=HDS}
/*_____*/

        // How to print HashMap
        //(1)//
        Set set=hm1.entrySet();
        System.out.println(set);
        //[70=DJU, 40=DJU,ARP, 60=DJU, 30=PAT, 15=HDS]
        Iterator itr=set.iterator();
        while(itr.hasNext())
        {
            Map.Entry entry=(Map.Entry) itr.next();
            System.out.println(entry.getKey()+"="+entry.getValue());
        }

        //(2)//

        for (Map.Entry m : hm1.entrySet())
        {
            System.out.println(m.getKey()+"="+m.getValue());
        }

        /*70=DJU
        40=DJU,ARP
        60=DJU
        30=PAT
        15=HDS */
    }
}

```